IdeaGen: An Evidence-Grounded Platform for Research-Gap Discovery with Local Language Models

Sanket Kumar
Dept. of Computer Science and Engineering (Data Science)
Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, India

Divyanshu Kumar
Dept. of Computer Science and Engineering (Data Science)
Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, India

Aditya Pratap Singh
Dept. of Computer Science and Engineering (Data Science)
Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, India

Abstract*—Identifying research gaps requires reading many papers, recording what each one reports as a limitation or open problem, and checking whether the same issue recurs across papers. This paper presents IdeaGen, a research-assistance platform that automates parts of this workflow while keeping every output traceable to source text. IdeaGen ingests papers from OpenAlex, downloads open-access PDFs where available, segments them into page-referenced passages, and uses a locally hosted language model (qwen3:8b served by Ollama) to extract typed research statements linked to their source passages. Statements are embedded with nomic-embed-text, grouped by average-linkage agglomerative clustering on cosine distance, and screened by rule-based pattern analysis to form candidate gaps. Each candidate is assessed by the language model under a JSON schema, and every statement identifier the model cites is checked against the statements actually placed in its prompt before the gap is stored. A lifecycle manager starts, reuses and stops the local inference server safely. In a team-recorded run over 127 statements, all twelve validation requests timed out after 300 s, and the validator stored them as scientific rejections. We describe the corrections: a bound on generated tokens, a bounded and deduplicated evidence set, and propagation of provider failures as HTTP 502 errors. The offline backend test suite passes (847 passed, 24 skipped, two live-model tests deselected), and a direct call to the validator with the real model, outside the API, returned a schema-valid verdict in 19.73 s. A full API-level re-run of gap discovery after these fixes is still pending, and the scientific quality of the discovered gaps has not been evaluated against expert judgement.

Keywords—research-gap discovery, large language models, retrieval-augmented generation, sentence embeddings, agglomerative clustering, evidence provenance, structured outputs, local inference

I. Introduction

A research gap is a question, limitation or direction that the existing literature identifies but does not resolve. Finding such gaps is a routine but laborious part of research: a student or researcher must read a set of papers, note the limitations and future-work statements each one reports, and judge whether the same problem appears in several papers or only in one. The work grows with the size of the reading list, and the reasoning behind a proposed gap is often recorded informally, which makes it hard for a supervisor or reviewer to check. IdeaGen makes this definition operational through three of the nine statement types it extracts: LIMITATION, FUTURE_WORK and PROBLEM. A candidate gap can only be formed from statements of these types.

Large language models (LLMs) can summarise and extract information from text, but they also produce fluent statements that are not supported by their input [1]. Retrieval-augmented generation (RAG) reduces this risk by conditioning generation on retrieved documents [2], and later work asks models to cite the passages that support each claim [3]. Several recent systems use LLMs to propose research ideas directly from literature [4]–[6]. These systems show what is possible. However, Si *et al.* also report that LLMs are unreliable when used to evaluate research ideas [5], so a model's verdict on an idea cannot serve as evidence by itself.

IdeaGen takes a narrower and more checkable approach. It does not ask a model to invent ideas. Instead, it extracts explicit statements from the papers in a project, groups statements that discuss the same topic, and proposes a candidate gap only where the papers themselves report limitations, future work or unresolved problems. A language model then assesses each candidate against its evidence. Application code discards any statement identifier the model cites that was not in its prompt, and accepts a VALID verdict only if at least one cited identifier survives. Because every statement is a database record with an identifier, page range and source passage, a reviewer can check each stored gap against the sentences it came from; a freely generated idea has no such anchor. In the default configuration all model inference runs on a local Ollama server, and only bibliographic requests and open-access PDF downloads leave the machine. A Gemini provider is implemented as a configurable alternative but was not used in the runs reported here.

The main contributions of this project are:

- a paper-understanding pipeline that turns PDF passages into typed research statements, each linked to its source passage, page range and section;
- a gap-discovery pipeline that combines statement embeddings, agglomerative clustering, deterministic pattern rules and schema-constrained LLM validation with an evidence-integrity check;
- a lifecycle manager for the local inference server that handles startup, readiness, model checks, concurrent use and safe shutdown; and
- a documented reliability investigation in which infrastructure timeouts were being recorded as scientific rejections, together with the corrective changes and their test evidence.

The scope is deliberately limited. IdeaGen proposes gaps relative to the papers in a project; it does not search all published literature, and it cannot establish that a gap is novel. Its outputs are candidates for a researcher to review, not verified scientific findings. The same rule is built into opportunity generation: the prior-art prompt forbids the model from claiming that an idea is definitely novel, and the default assessment text is “Potentially novel within the analyzed corpus.”

II. Problem Statement and Objectives

A. Problem Statement

Given a project corpus of research papers, the system should produce a set of candidate research gaps. Each gap must be supported by statements extracted from papers in that corpus, and each statement must be traceable to the passage it came from. When the model or its server fails, the failure must be reported as a failure and must not be stored as a judgement about the science. In the implementation, the corpus is the set of papers linked to a project through the project_papers table, and each output is a research_gaps record whose verified evidence is stored as gap_evidence records.

B. Functional Objectives

- Ingest papers from OpenAlex or PDF upload and split their text into ordered, page-referenced passages.
- Extract typed statements (problem, method, finding, limitation, future work and others) with passage provenance.
- Group related statements across papers and derive candidate gaps from recurring limitations and open directions.
- Validate candidates with an LLM and keep only evidence that can be verified against stored statements.
- Persist papers, statements, gaps and evidence links, and expose them through a REST API and a web interface.

C. Non-Functional Requirements

Each requirement below is followed by the mechanism that implements it. *Provenance:* every stored gap must link to statements, papers and passages that exist; this is enforced by foreign keys from gap_evidence to research_statements and papers, and from statements to passages. *Data integrity:* the model must not be able to introduce evidence identifiers that it was not given; cited identifiers are checked against the prompt's evidence pool. *Reliability:* provider failures must fail visibly; they raise ProviderFailureError, stop discovery and return HTTP 502. *Bounded latency:* every model call is limited in input (at most 12 passages and 2,500 words for extraction; at most three statements with 800-character excerpts for validation), output (512 tokens) and time (300 s timeout). The input and output bounds for validation were added only in Phase 5D, after this requirement was found to be unmet (Section VII). *Local execution:* the default providers call a local Ollama server. *Testability:* providers are passed into the pipelines, so tests replace them with deterministic fakes.

III. Related Work

A. Grounded Generation

Ji *et al.* survey hallucination in natural-language generation and its causes [1]. Lewis *et al.* introduced RAG, which combines a parametric generator with a non-parametric document index [2]. Gao *et al.* evaluate whether LLM answers cite passages that actually support them [3]. IdeaGen borrows the principle that claims should be tied to retrieved evidence, but it applies the check deterministically: cited statement identifiers are compared with the set the model was given.

B. LLM-Assisted Research Ideation

ResearchAgent iteratively proposes problems, methods and experiments from a core paper and its related literature, refined by LLM reviewing agents [4]. The AI Scientist automates idea generation, experiments and paper writing [6]. In a large human study, Si *et al.* found that LLM-generated ideas were judged more novel than expert ideas but slightly weaker on feasibility, and they report that LLM self-evaluation is unreliable [5]. IdeaGen is closer to evidence synthesis than to open-ended ideation: it reports gaps that papers already state, with their provenance, and treats the model's verdict as one check among several.

C. Embeddings, Vector Search and Clustering

Sentence-BERT showed that sentence embeddings compared by cosine similarity support efficient semantic search and clustering [7]. IdeaGen uses nomic-embed-text, an openly trained long-context text embedder [8], served locally. FAISS provides exact and approximate nearest-neighbour search over dense vectors [9]; IdeaGen uses its exact inner-product index. Hierarchical agglomerative clustering with average linkage is a standard method for grouping documents without fixing the number of clusters in advance [10]; IdeaGen uses the scikit-learn implementation [11].

D. Structured Outputs and Local Inference

Constraining generation to a grammar or JSON schema makes model output machine-parseable [12]. Ollama exposes this through the `format` field of its chat API, which accepts a JSON schema [13]. The chat model is qwen3:8b from the Qwen3 family, which supports separate thinking and non-thinking modes [14]; IdeaGen requests the non-thinking mode. IdeaGen's contribution is not a new model or algorithm. It is the integration of these established components into a pipeline with explicit provenance, validation and failure-handling rules.

IV. Proposed System and Architecture

Fig. 1 shows the implemented architecture. Paper metadata is obtained from OpenAlex, an open index of scholarly works [15]. A React and TypeScript single-page application calls a FastAPI [16] backend over HTTP. The backend contains the ingestion, understanding, gap-discovery and opportunity modules, a workflow engine that executes research runs, a provider layer that isolates all Ollama-specific code, and a persistence layer built on SQLAlchemy [17]. Request and response models, including the schemas sent to the language model, are Pydantic classes [18]. Table I summarises the technology stack.

TABLE I. Technology Stack and Responsibilities

Layer	Technology	Responsibility
Frontend	React 19, Vite, TypeScript	Projects, paper search, runs, gap and evidence views
API	FastAPI, Pydantic	REST endpoints, request validation, error mapping
PDF text	PyMuPDF [19]	Text and layout extraction
Chat model	qwen3:8b via Ollama	Statement extraction, gap validation
Embeddings	nomic-embed-text, 768-d	Statement vectors
Vector index	FAISS IndexFlatIP	Exact cosine search (prior-art retrieval)
Clustering	scikit-learn	Agglomerative clustering
Storage	SQLAlchemy, Alembic	SQLite in tests and demo; PostgreSQL target

A project groups the papers a researcher has added. A research run executes three stages in a background thread: ingestion, understanding and gap discovery. A run that discovers gaps pauses in the AWAITING_REVIEW state; generating a research opportunity from a gap requires a separate, explicit request. Runs end as COMPLETED, PARTIAL_SUCCESS or FAILED, and a run interrupted by a process crash is marked INTERRUPTED and can be retried. Within a run, gap discovery and prior-art retrieval read only the statements of the project's own papers. Table II lists the endpoints most relevant to this report.

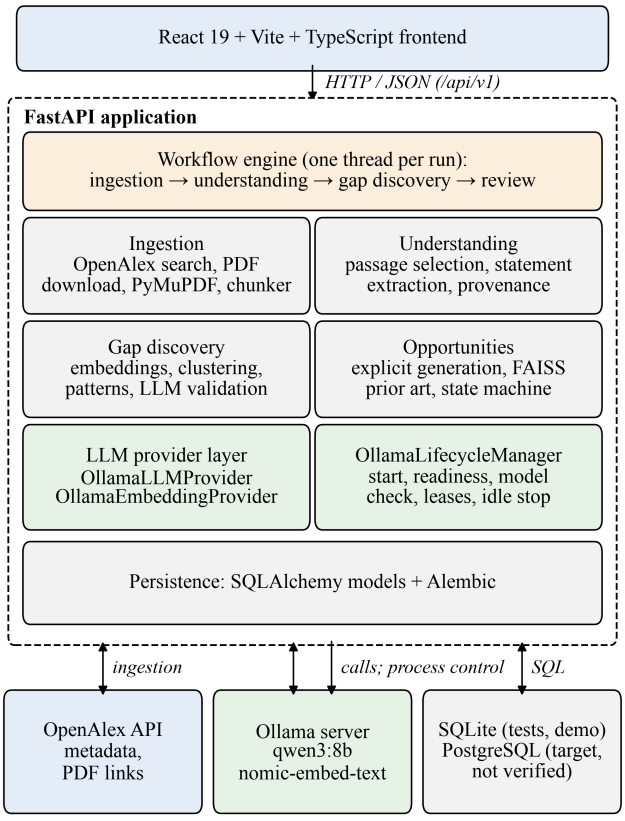


Fig. 1. Implemented IdeaGen architecture. Arrows to external services show the module that uses each service.

TABLE II. PRINCIPAL API ENDPOINTS

Endpoint (prefix /api/v1)	Behaviour
GET /search/openalex	Query OpenAlex; nothing is stored
POST /papers/ingest/openalex	Store metadata, download open-access PDF, create passages
POST /papers/{id}/understand	Extract statements; 404 if paper or passages missing; 502 on model failure
POST /gaps/discover	Run gap discovery, optionally scoped by project_id and a publication cutoff; 502 on provider failure
GET /gaps, /gaps/{id}	List or read stored gaps with evidence identifiers
POST /projects/{id}/runs	Start a research run; 400 for invalid stages, 409 if a run is active
GET /system/status	Report configured models and live Ollama reachability

The canonical routes use the /api/v1 prefix; several routes also have legacy aliases under /api, for example /api/papers/{id}/understand and /api/gaps/discover, which call the same handlers.

V. METHODOLOGY AND IMPLEMENTATION

A. Paper Ingestion and Passage Segmentation

Papers are added by OpenAlex identifier or DOI. An upload endpoint (/api/papers/upload, at most 20 PDFs per request) runs the same text extraction and chunking, but the current web interface does not use it. The OpenAlex path stores the work's metadata and, where an open-access PDF exists, downloads it through a hardened client that accepts only public destinations, validates redirects and checks the file size and PDF signature. If no PDF is available the paper is stored as metadata only. PyMuPDF extracts text blocks, a section detector assigns headings, and a deterministic chunker merges blocks into passages with a target of 1,000 characters (minimum 150, maximum 1,600). Each passage records its index, page range and section. Files are identified by a SHA-256 content hash, and a missing publication date is stored as null rather than estimated.

B. Statement Extraction and Provenance

The understanding stage first selects a bounded set of passages: at most 12 passages and 2,500 words by default. When a paper exceeds this budget, the selector prefers the abstract, then limitation and future-work sections, then the introduction and conclusion, then methods, results and discussion, and finally restores document order. Each selected passage is given to the model with a header of the form [PASSAGE i | Pages a–b | Section: s].

The model must return an object that conforms to a Pydantic schema. The schema contains the research problem, questions, methodology, datasets, findings, limitations and future work, and a list of statements. Each statement has a type from nine categories (PROBLEM, QUESTION, METHOD, DATASET, FINDING, LIMITATION, FUTURE_WORK, ASSUMPTION, CONTRIBUTION) and the index of the passage that supports it. Application code then resolves provenance. An index that matches a selected passage is linked to that passage's identifier, pages and section. An index that does not match is left unlinked, so that a wrong index cannot produce a false attribution. If the model writes the index inside the statement text, the reference is removed from the text and recovered; if no index is given, an exact substring match against the selected passages is attempted. If extraction fails, a FAILED knowledge record is kept for audit, no statements are stored, and the endpoint returns HTTP 502.

C. Embedding Generation

Statements are embedded through Ollama's /api/embed endpoint with nomic-embed-text [8]. The provider checks that every vector has the configured dimension (768) and normalises each vector to unit length. If the batch endpoint is unavailable it falls back to the older per-text endpoint. Sentence-transformer and Gemini embedding providers are also implemented and can be selected by configuration, but the default and tested configuration uses Ollama.

D. Similarity Index

For unit-length vectors the inner product equals cosine similarity:

$$s(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|},\ d(\mathbf{u},\mathbf{v}) = 1 - s(\mathbf{u},\mathbf{v}) \qquad (1)$$

where **u** and **v** are statement embeddings and d is the cosine distance. Each discovery run rebuilds an exact FAISS inner-product index (IndexFlatIP) from the loaded statements [9]. The index is written to disk only when discovery covers the whole database without a date cutoff, so a scoped or time-restricted run cannot overwrite the shared index. The index is a derived structure: the database remains the source of truth, and the index can always be rebuilt. It is used for prior-art retrieval during opportunity generation (top 10 statements, minimum similarity 0.2). Clustering operates on the embedding matrix directly.

E. Semantic Clustering

Statements are grouped with agglomerative clustering using cosine distance and average linkage [10], [11]. The distance between clusters A and B is the mean pairwise distance

$$D(A,B) = \frac{1}{|A||B|}\sum_{a\in A}\sum_{b\in B} d(a,b) \qquad (2)$$

and two clusters are merged only while

$$D(A,B) < \tau,\ \tau = 1-\theta = 1-0.65 = 0.35 \qquad (3)$$

where θ is the similarity threshold. The number of clusters is therefore determined by the data. When more than one cluster is formed, clusters with fewer than two statements are discarded as noise. The threshold of 0.65 is a fixed default; it has not been tuned on labelled data.

F. Pattern Analysis and Candidate Generation

Fig. 2 shows the complete discovery workflow. Within each cluster, statements are grouped by type. A LIMITATION statement is accepted only if it describes a deficit: a set of regular expressions looks for deficit terms (for example "fails", "limited", "latency" or "hallucinat") and rejects statements that only assert sufficiency, feasibility or superior performance. A cluster yields at most one candidate, and only if it contains an accepted limitation, a future-work statement or a problem statement. The candidate's text is built from a template around the first matching statement; no language model is involved at this step. The candidate carries the identifiers of its supporting statements and papers, together with counts of each statement type. Each candidate is labelled with one of five patterns:

- LIMITATION_AND_FUTURE_WORK: accepted limitations and future-work statements in one cluster;
- RECURRING_LIMITATIONS: limitations reported by more than one paper;
- DOCUMENTED_LIMITATION: limitations from a single paper;
- OPEN_FUTURE_WORK: future-work statements without accepted limitations; and
- UNRESOLVED_PROBLEM: problem statements only.

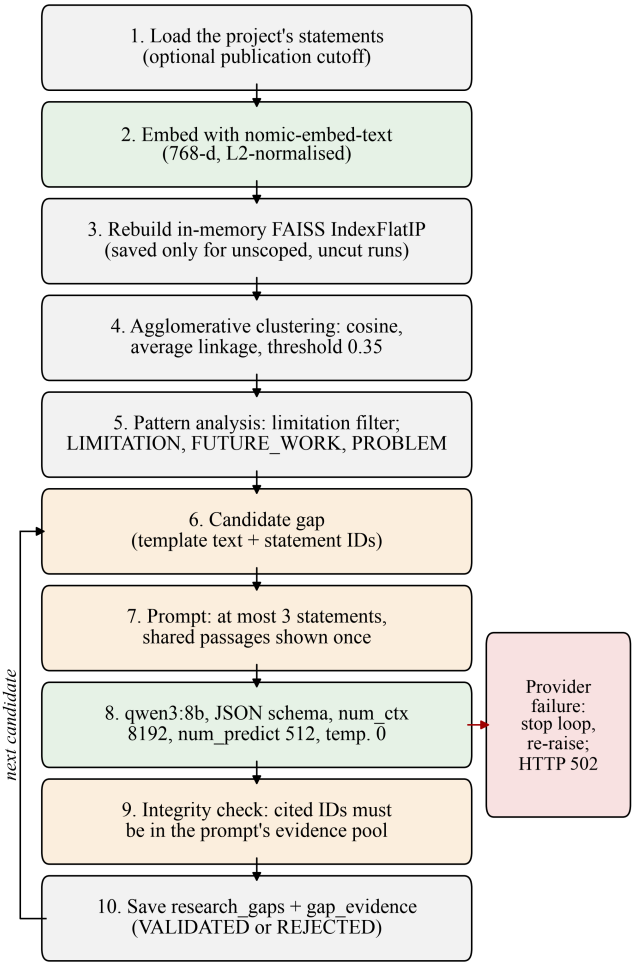


Fig. 2. Gap-discovery workflow as implemented in the discovery pipeline. Candidates are validated and stored one at a time; a provider failure stops the loop.

G. LLM-Based Validation

Each candidate is sent to qwen3:8b with a reviewer prompt that contains the gap statement, the research problem, the reason existing work is insufficient, and an evidence block. The model must answer with a verdict (VALID or REJECT), its reasoning, the identifiers of supporting and counter-evidence statements, and an optional confidence in [0, 1]. Each evidence item is introduced by a line of the form [ID: statement UUID | Paper: paper UUID | Type | Section], followed by the statement and its passage excerpt. These UUIDs are the primary keys of the stored statements, and the prompt is the only place the model can obtain them. The prompt instructs the model that every cited identifier must be one of those listed.

H. Evidence Selection and Deduplication

Each validation prompt contains at most three supporting statements. Each statement is listed with its identifier, paper, type, section and an excerpt of its source passage of at most 800 characters. When two statements come from the same passage, the passage text is shown only once, and the later statement refers back to the first. If a statement's source passage cannot be found, the prompt says so explicitly instead of supplying text. When only part of a candidate's evidence is evaluated, the stored gap records both the total and the evaluated evidence counts.

This bound keeps prompts short and predictable, but it has a cost. The three statements are the first three in the candidate's order, not the three most relevant, and evidence that is never shown to the model cannot influence the verdict. The pattern analyser orders supporting statements as limitations first, then future-work statements, then problem statements. In a LIMITATION_AND_FUTURE_WORK cluster with three or more accepted limitations, the validator therefore never sees the future-work statements that gave the pattern its name. A validated gap also stores at most three evidence statements by default. Validation therefore assesses whether the selected evidence supports the candidate, not whether the candidate is supported by all available evidence. Before Phase 5D, every supporting statement was placed in the prompt, which removed this trade-off but made prompt length grow with cluster size.

I. Evidence-Integrity Checks

The model's verdict is not accepted as given. Each cited identifier is parsed as a UUID and kept only if it belongs to the evaluated evidence pool; duplicates are removed. A VALID verdict becomes a VALIDATED gap only if at least one cited identifier survives this check. Only verified statements are written as gap evidence. Otherwise the gap is stored as REJECTED, with one of three reason codes in its metadata:

- EMPTY_EVIDENCE_CITATIONS: the model cited no statements;
- INVALID_OR_FABRICATED_CITATIONS: no cited identifier could be verified; or
- INSUFFICIENT_EVIDENCE_COUNT: fewer verified statements remain than required.

The surviving identifiers become gap_evidence records with foreign keys to research_statements and papers. As a result, the stored evidence of a VALIDATED gap is always a subset of the statements the model was shown, and each one can be traced back to its passage. The model's raw verdict is kept in the gap's metadata (model_verdict), so a VALID verdict that failed the integrity check remains visible.

J. Persistence and API Behaviour

Gaps are written one at a time, immediately after validation. The project and run identifiers are attached when each gap is saved, so gaps saved before a later failure still belong to their project. An unknown project returns HTTP 404. Exceptions that derive from the LLM error hierarchy, or embedding provider errors, are mapped to HTTP 502 (bad gateway), because the fault lies in an upstream service; other exceptions return HTTP 500.

K. Ollama Provider and Lifecycle Management

All model calls go through a provider interface with a single method, generate_structured, which takes a prompt and a Pydantic class and returns a validated instance of that class. The Ollama provider sends a non-streaming request to /api/chat with the class's JSON schema as `format`, `think` set to false, temperature 0, a context window (`num_ctx`) of 8,192 tokens and, by default, an output limit (`num_predict`) of 512 tokens [13]. The request timeout defaults to 300 s. Code fences around the JSON are removed before parsing. Timeouts, connection errors, non-200 responses, unparseable HTTP bodies and empty replies raise ProviderFailureError; content that is not valid JSON or does not match the schema raises MalformedResponseError.

The output limit is configured once and shared. The setting ollama_num_predict is read by get_default_llm_provider(), and both the paper-understanding pipeline and the gap-discovery pipeline call this factory when no provider is passed in. The 512-token limit therefore applies to statement extraction as well as to validation. Before Phase 5D no num_predict value was sent, so the Ollama server's own default applied.

A local inference server is a separate process that holds gigabytes of model weights in memory, so it should run only while needed, should be ready before the first request, and must not be stopped while another request is using it. The OllamaLifecycleManager, shown in Fig. 3, implements these rules. Each provider call takes a lease. Acquiring a lease cancels any pending shutdown and checks whether the server answers on /api/tags. If it does, the server is reused and marked as not owned. If it does not and automatic management is enabled, the manager starts `ollama serve` and polls until the server is ready or the 30 s startup timeout expires. Required models are then checked against the installed list; missing models raise an error with an `ollama pull` hint and are never downloaded automatically. Releasing the last lease starts a 60 s idle timer if, and only if, the manager owns the process. When the timer fires, the manager re-checks the lease count and ownership and terminates only its own child process, killing it if it has not exited after 5 s. Application shutdown is conservative and skips termination while leases remain; an exit hook forces cleanup of an owned process when the Python process ends.

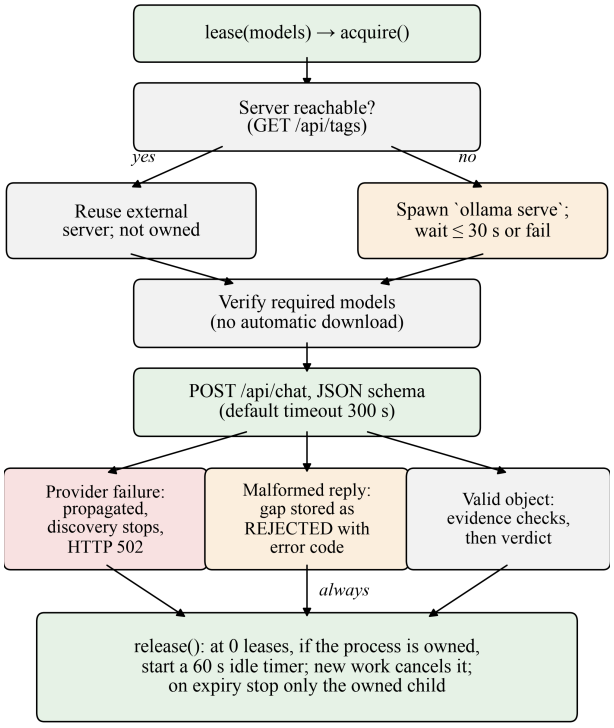


Fig. 3. Ollama lease lifecycle and the classification of validation outcomes.

L. Opportunity Generation and Lifecycle

A validated gap can be turned into a research opportunity on explicit request. Generation retrieves related statements from the project as prior art and asks the model for a proposed

direction and an assessment. Opportunities then follow a deterministic state machine with four states (OPEN, STRENGTHENED, PARTIALLY_ADDRESSED, RESOLVED). The model classifies how a newer paper relates to an opportunity; application code alone applies the state transition, and RESOLVED requires at least two addressing papers. This component is implemented and unit-tested but is outside the experiments reported here.

VI. Database Design and Evidence Traceability

Fig. 4 shows the tables that carry provenance. The schema is managed by a linear chain of 13 Alembic revisions; JSON fields use JSONB on PostgreSQL and JSON on SQLite. A paper has many passages and at most one research-knowledge record, enforced by a unique key. Each research statement references its knowledge record and paper and, optionally, its passage; the statement also stores its own page range and section. A research gap stores its statement, status, evidence count, validation reasoning and a JSON metadata field that holds the pattern signals, the model verdict, any rejection code, the evaluated-evidence counts and the run identifier. The gap_evidence table links each gap to the statements and papers that were verified for it.

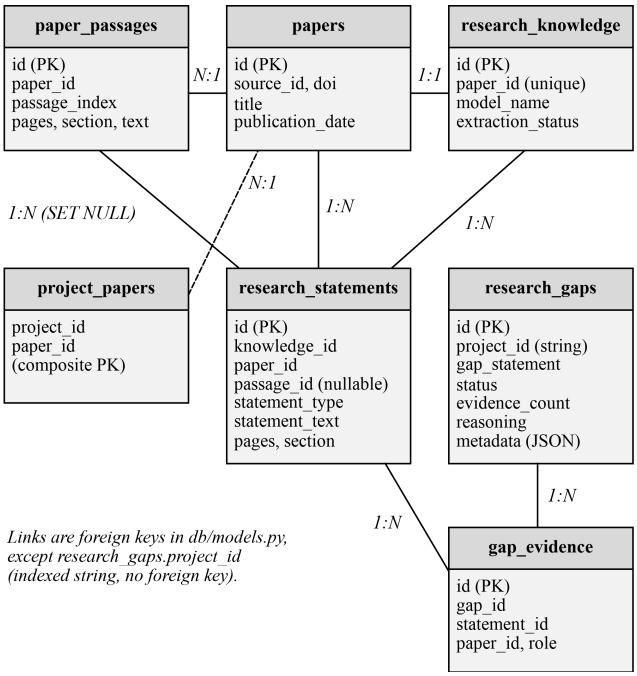


Fig. 4. Tables involved in statement and gap provenance, as declared in the SQLAlchemy models.

These links make a gap auditable. From a gap, a reader can follow gap_evidence to each verified statement, from the statement to the exact passage and page range, and from the paper to its DOI or OpenAlex identifier. The design has two weaknesses. First, research_gaps.project_id is an indexed string without a foreign key, so project scoping is enforced by application code. Second, deleting a passage sets the statement's passage reference to null; the stored page range and section survive, but the passage text does not.

VII. Experimental Setup and Results

A. Environment and Evidence Sources

All work was done on a local macOS machine with a Python 3.14.8 virtual environment, a local Ollama server and the models qwen3:8b and nomic-embed-text. Tests and experiments used SQLite databases; PostgreSQL was not used in any reported run. Hardware specifications were not recorded with the experiments, so the timings below indicate scale only and are not benchmarks. Evidence comes from three sources, and each table states its source: (a) test commands executed during preparation of this report on 23 October 2026, with no model inference; (b) records in the repository; and (c) results recorded by the team during development whose raw logs are not archived in the repository.

We distinguish four levels of evidence. *Unit tests* exercise code with mocked services. A *direct live-model* test calls an application component with the real model but without the HTTP API. An *API integration* run sends a request to an endpoint, which uses the real model and writes to a database. A *full-workflow* run executes the multi-stage pipeline, either through the research-run engine or the experiment runner. A result at one level does not establish the levels above it.

B. Automated Tests

The backend test suite uses pytest. Ollama, OpenAlex and PDF downloads are mocked, and a guard in the test configuration aborts the run if it is pointed at a project database under data/. Table III lists the results. The focused command covers configuration, the Ollama provider, the gap-discovery pipeline and the lifecycle manager; its result matches the 98 passed and 2 skipped recorded at the end of Phase 5D. The 21 lifecycle tests cover startup and readiness, immediate process exit, bounded startup polling, external-server reuse, disabled auto-management, missing models without downloads, model-tag matching, lease counting under exceptions and concurrency, idle shutdown and its cancellation, a race between new work and the idle timer, never terminating an external process, terminating only the owned child, and conservative shutdown.

TABLE III. Automated Test Results

Suite	Result	Source
Focused: config, provider, gap discovery, lifecycle (4 files)	98 passed, 2 skipped, 0 failed (3.89 s)	Re-executed 23 Oct 2026
Full backend suite, live-model tests deselected	847 passed, 24 skipped, 2 deselected, 0 failed (22.93 s)	Re-executed 23 Oct 2026
Headless-browser end-to-end checks	29 passed, 0 failed	Reliability audit record, 23 Oct 2026

The skipped tests require a PostgreSQL database or a reachable Ollama server; neither was available to the test runs. The two deselected tests call a live model and were excluded on purpose. These results show that the code behaves as specified with mocked services. They do not show that the model's outputs are correct.

C. Live-Model Observations

Table IV collects the runs in which the real models were used. The paper-understanding run used the canonical endpoint POST /api/v1/papers/{paper_id}/understand on an isolated database and created a successful knowledge record with 22 statements, each with passage provenance. The five-paper experiment is documented in the repository with its configuration and artifacts; it used a compact budget of four passages and 600 words per paper. These runs show that the components work together on small corpora. They do not establish the accuracy or completeness of the extracted statements. All runs in Table IV except the last took place

before the Phase 5D changes, so none of them tests the corrected code at the API or workflow level.

TABLE IV. LIVE-MODEL OBSERVATIONS

Run	Observation	Source
Paper understanding, 1 paper (API integration)	HTTP 200 in 84.66 s; 22 statements with passage provenance	Team record (not archived)
Five-paper experiment, 19 Oct 2026 (full workflow, runner)	502 passages; 63 statements in 283.86 s; 1 validated gap; 1 opportunity	Repository report and artifacts
Audit research run, 23 Oct 2026 (full workflow, run engine)	18 statements; 2 gaps validated; explicit opportunity in 41.6 s	Reliability audit record
Gap discovery, 127 statements (discovery pipeline)	12 candidates; every validation call timed out at 300 s	Team record (not archived)
Single-candidate validation, Phase 5D (direct live-model)	Schema-valid VALID verdict in 19.73 s	Team record (not archived)

D. Initial Gap-Discovery Failure

The 127-statement run was the first gap discovery on a corpus of realistic size. It produced 12 candidates, and the validation request for each one timed out after 300 s. Because validation is sequential, this amounts to at least 3,600 s of waiting. The committed version of the validator (commit 945645a) shows how this happened. It placed every supporting statement of a candidate in the prompt, each with a passage excerpt of up to 800 characters, and repeated the excerpt when two statements shared a passage. It sent no output-length limit. Its exception handler caught ProviderFailureError together with all other exceptions and returned a REJECTED gap with the code MODEL_INFERENCE_ERROR and the reasoning "Validation failed due to provider error." All 12 candidates were therefore stored as rejections, although the model had not returned any judgement. The prompt sizes of that run were not recorded, so whether input length, output length or server load was the dominant cause of the timeouts was not isolated.

This behaviour is harmful for three reasons. A REJECTED status tells a researcher that the evidence does not support the gap, which was not established. The stored rejections are indistinguishable from genuine ones in any list filtered by status. And the run continued through every candidate after the first timeout had already shown the server to be unusable.

E. Phase 5D Corrections

The changes target the two quantities under the application's control that determine how long a call takes, input length and output length, together with the handling of failures. Table V maps each change to its implementation and tests. All three were confirmed in the current source during preparation of this report.

TABLE V. PHASE 5D CORRECTIVE CHANGES

Problem	Change	Test evidence
Unbounded output length	Setting ollama_num_predict (default 512) passed through the provider factory to the num_predict option	Config default; option present, custom and omitted
Long, repetitive prompts	At most 3 statements per candidate; repeated passages rendered once; total and evaluated counts recorded	Prompt bounding and passage deduplication tests
Timeouts stored as rejections	ProviderFailureError re-raised; pipeline stops; API maps to HTTP 502	Validator propagation; pipeline stops after first failure

The HTTP 502 mapping for POST /gaps/discover was confirmed by reading the handler and its shared error classifier. A test asserts the 502 response for the understanding endpoint, but no test currently asserts it for the discovery endpoint.

F. Single-Candidate Live Validation

After the changes, one candidate was validated directly through the application's provider, lifecycle manager and gap validator, using the locally installed qwen3:8b. The database was opened read-only and no gap was persisted. Table VI lists the recorded measurements. Every statement identifier the model cited was found in the candidate's evidence pool. This is a single run; no repetitions, accuracy figures or confidence intervals are available, and the run was not repeated for this report.

TABLE VI. SINGLE-CANDIDATE VALIDATION MEASUREMENTS

Quantity	Value
Prompt length	3,938 characters
Statements evaluated / unique passages	3 / 2
num_ctx / num_predict	8,192 / 512 tokens
Inference time	19.73 s
Output	Schema-valid; verdict VALID
Cited identifiers in evidence pool	Yes (all)
Database access	Read-only; nothing persisted

G. Pending End-to-End Validation

A bounded API-level test of the complete discovery workflow after the Phase 5D changes (Phase 5E) was planned, but its execution failed with a tooling error before the test ran. It has not been repeated. The successful direct validator call therefore does not show that the full sequence of API request, embedding, clustering, validation of several candidates and database writes now completes within acceptable time. Table VII summarises the verification status of the main capabilities.

TABLE VII. VERIFICATION STATUS OF MAIN CAPABILITIES

Capability	Status	Evidence
Ingestion, passages, provenance	Implemented	Unit and API tests; used in live runs
Statement extraction	Implemented	Live runs recorded before the 512-token output limit
Clustering and candidate generation	Implemented	Unit tests with deterministic embeddings
Bounded LLM validation	Implemented	Unit tests; one direct live-model call
HTTP 502 on provider failure (discovery)	Implemented	Code inspection only
Ollama lifecycle manager	Implemented	21 unit tests with mocked processes
Full discovery API after Phase 5D	Pending	Phase 5E not completed
PostgreSQL deployment	Not verified	Schema and migrations exist
Scientific quality of gaps	Not evaluated	No labelled ground truth

VIII. RELIABILITY, SAFETY AND ERROR HANDLING

Rejection versus failure. A scientific rejection means that the model returned a schema-valid verdict and the outcome was REJECTED, either because the verdict was REJECT or because a VALID verdict failed the evidence checks. Both cases are judgements about the evidence, and the stored metadata records the model's verdict and any rejection code. A provider failure means that no verdict was obtained: a timeout (300 s by default), a connection error, an HTTP error or an empty reply. Since Phase 5D such failures stop discovery and are never stored as gaps. *Structured output.* Model output is parsed into Pydantic classes; field types, enumerations and the [0, 1] range of confidence values are enforced. *Fail-fast behaviour.* The first provider failure ends the discovery loop instead of repeating a 300 s timeout for each remaining candidate.

Database isolation. The test suite refuses to run against a SQLite file inside the project's data directory, and the reported tests used temporary databases. The live validation in Section VII opened its database read-only. *Process ownership.* The lifecycle manager never terminates a server it did not start, and it terminates only the exact child process it created.

Some gaps remain. A malformed response, or any other unexpected exception during validation, is still stored as REJECTED with the MODEL_INFERENCE_ERROR code. The code keeps such records distinguishable in metadata, but the status alone still conflates them with scientific rejections. There is no automatic retry. Gaps validated before a provider failure remain stored; this is intended, and they are tagged with their run. Finally, an embedding response that is empty or has the wrong dimension raises a generic error and is reported as HTTP 500 rather than 502.

IX. DISCUSSION AND LIMITATIONS

Latency. Local inference with an 8-billion-parameter model is slow: tens of seconds per call in the recorded runs, and discovery calls the model once per candidate. All model calls run inside the HTTP request or the run thread; there is no job queue. *Model dependence.* Extraction quality, validation verdicts and output formatting all depend on one model and one prompt per task.

Context and output limits. The extraction prompt can include up to 2,500 words of passages within an 8,192-token context window. The 512-token output limit is applied to every call made through the default provider, including statement extraction. A long extraction response may be cut off and then fail schema validation. The extraction runs in Table IV were recorded before this limit was introduced, and its effect on extraction has not been measured. *Evidence subset.* Validation sees at most three statements, chosen by position rather than relevance.

False positives and negatives. The limitation filter is a keyword heuristic, so it can admit statements that are not real limitations and reject ones that are. Gap text is templated from a single statement and may describe the cluster poorly. Clusters with one statement are discarded, so a gap reported in only one sentence of the corpus can be missed. The similarity threshold has not been tuned. *Corpus dependence.* IdeaGen sees only the papers in the project, and only the passages selected from them. A gap it reports may already be solved in a paper outside the project. It performs no external literature search, so it cannot establish novelty.

Evaluation. There is no labelled ground truth for gap discovery, and no comparison with human reviewers or baselines on real data. The related relationship classifier used for opportunity evolution reached 0.68 accuracy on 25 hand-labelled cases, but its prompt was revised after inspecting errors on those same cases, so this is not a held-out estimate. *Deployment.* PostgreSQL behaviour, Python versions other than 3.14, authentication and multi-user isolation are not verified or not implemented.

X. CONCLUSION AND FUTURE WORK

IdeaGen implements a complete path from paper ingestion to stored, evidence-linked research-gap candidates, using local models for extraction, embedding and validation. Its main engineering features are passage-level provenance, deterministic checks on the evidence a model cites, and a lifecycle manager that runs the local inference server safely. Testing with real models exposed a defect in which server timeouts were recorded as scientific rejections. The Phase 5D changes bound model output and prompt size and make provider failures stop the pipeline with an HTTP 502 error. These changes pass the focused and full offline test suites, and one direct validator call with the real model, outside the API, succeeded in 19.73 s.

The following work is not yet done. The bounded end-to-end discovery test (Phase 5E) must be run through the API and its timing and stored records examined. The effect of the 512-token limit on statement extraction should be measured, with a separate limit for extraction if needed. Evidence for validation should be ranked by relevance rather than position, and the malformed-response path should receive a status distinct from REJECTED. Long model calls should move to background jobs. Beyond engineering, the system needs an evaluation dataset with expert-labelled gaps, comparison against simple baselines, tuning of the clustering threshold, verification on PostgreSQL, and a search of literature outside the project before any gap is described as open.

REFERENCES

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.

[3] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023.

[4] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang, “ResearchAgent: Iterative research idea generation over scientific literature with large language models,” in *Proc. NAACL*, vol. 1, 2025, pp. 6709–6738.

[5] C. Si, D. Yang, and T. Hashimoto, “Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[6] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, “The AI Scientist: Towards fully automated open-ended scientific discovery,” 2024, arXiv:2408.06292.

[7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[8] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic Embed: Training a reproducible long context text embedder,” 2024, arXiv:2402.01613.

[9] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[10] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[11] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] B. T. Willard and R. Louf, “Efficient guided generation for large language models,” 2023, arXiv:2307.09702.

[13] Ollama, “Ollama API documentation.” [Online]. Available: https://github.com/ollama/ollama/blob/main/docs/api.md

[14] A. Yang *et al.*, “Qwen3 technical report,” 2025, arXiv:2505.09388.

[15] J. Priem, H. Piwowar, and R. Orr, “OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts,” 2022, arXiv:2205.01833.

[16] S. Ramírez, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[17] SQLAlchemy, “SQLAlchemy 2.0 documentation.” [Online]. Available: https://docs.sqlalchemy.org/en/20/

[18] Pydantic, “Pydantic documentation.” [Online]. Available: https://docs.pydantic.dev/

[19] Artifex Software, “PyMuPDF documentation.” [Online]. Available: https://pymupdf.readthedocs.io/